

SPIMOE: Exploiting Hybrid Sparsity for Reasoning MoE Inference on Heterogeneous PIM Architectures*

Rubing Yang¹, Cenlin Duan^{2†}, Yingjie Qi¹, Xiaolin He¹, Xiao Ma¹, Jianlei Yang^{1†}

¹School of Computer Science, Beihang University, Beijing, China

²School of Integrated Circuits and Systems, Beihang University, Beijing, China

Abstract

Long-reasoning Mixture-of-Experts (MoE) models expose two coupled inference bottlenecks: growing KV caches shift the critical path toward attention, while sparse expert activation causes load imbalance and low hardware utilization. Although Processing-in-Memory (PIM) offers a promising way to mitigate data movement overhead, existing PIM-based accelerators typically optimize attention or FFNs in isolation. We propose SPIMOE, the first co-design framework that exploits hybrid sparsity for efficient MoE inference on heterogeneous PIM architectures. SPIMOE combines adaptive expert routing with block-sparse attention and physical KV-cache eviction, and disaggregates attention and FFNs across SRAM-PIM and HBM-PIM. Static expert mapping and dynamic sub-batch scheduling further balance channel loads and overlap the two paths. Evaluations show that SPIMOE achieves up to 8.35× end-to-end speedup over an NVIDIA A100 GPU and 3.33× speedup in MoE FFN execution over PIMoE, while preserving reasoning accuracy comparable to full-attention baselines.

CCS Concepts

• **Computer systems organization** → **Heterogeneous (hybrid) systems**; • **Hardware** → **Emerging architectures**; • **Computing methodologies** → **Machine learning**.

Keywords

Processing-in-Memory, Mixture-of-Experts, Sparse Attention, Adaptive Routing

ACM Reference Format:

Rubing Yang, Cenlin Duan, Yingjie Qi, Xiaolin He, Xiao Ma, and Jianlei Yang. 2026. SPIMOE: Exploiting Hybrid Sparsity for Reasoning MoE Inference on Heterogeneous PIM Architectures. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD '26)*, November 08–12, 2026, San Jose, CA, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3831252.3834223>

*This work was supported by the National Natural Science Foundation of China (Grant No.62572036).

†Corresponding authors are Cenlin Duan and Jianlei Yang, Email: duancenlin@buaa.edu.cn, jianlei@buaa.edu.cn.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

ICCAD '26, San Jose, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2873-0/2026/11

<https://doi.org/10.1145/3831252.3834223>

Table 1: Comparison with prior PIM-based LLM acceleration works.

Work	Architecture	Algorithm	Focus Layer	Reasoning	Co-Optim.	KV Drop
Stratum [17]	NPU+DRAM-PIM	Topic Classify	MoE	×	×	×
Duplex [18]	xPU+DRAM-PIM	None	MoE	×	×	×
PIMoE [19]	NPU+DRAM-PIM	N:M Pruning	MoE	×	✓	×
TranCIM [20]	SRAM-PIM	Sparse Attn.	Attn.	×	✓	✓
H2EAL [21]	Hybrid-Bonding	Sparse Attn.	Attn.	×	✓	×
Ours	Hetero. PIM	Adaptive Routing + Sparse Attn.	MoE+Attn.	✓	✓	✓

1 Introduction

Large Language Models (LLMs) have achieved remarkable success across diverse domains, ranging from natural language understanding to autonomous code generation and scientific discovery [1–4]. To further scale model capacity without proportionally increasing computation, Mixture-of-Experts (MoE) has emerged as a promising architecture by activating only a subset of experts for each token [5–9]. However, the growing adoption of Chain-of-Thought (CoT) reasoning shifts LLM inference from short-response generation to long-chain reasoning [10–13], fundamentally changing the inference workload and exposing the limitations of conventional von Neumann-based accelerators. First, prolonged reasoning sequences incur a linear growth of the KV cache footprint, pivoting the system’s execution profile from being feed-forward network (FFN)-dominated to attention-dominated. Meanwhile, the dynamic expert routing mechanism in MoE introduces irregular, unpredictable accesses to expert weights, further aggravating data movement overhead. As a result, conventional accelerators struggle to efficiently support sparse long-reasoning MoE inference.

While Processing-in-Memory (PIM) [14–16] is promising for mitigating data movement overhead, directly mapping long-reasoning MoE workloads onto PIM architectures remains non-trivial. As summarized in Tab. 1, existing PIM-based accelerators [17–21] mainly optimize either MoE or attention in isolation, and therefore fall short of holistically supporting long-reasoning workloads. First, MoE-centric designs fail to exploit the temporal activation sparsity of experts for dynamic pruning, leading to redundant data movement. Second, current attention-centric optimizations often incur significant accuracy degradation in long-chain reasoning tasks and fail to alleviate the fundamental KV cache capacity bottleneck. Finally, the lack of Attention-FFN decoupling and holistic co-optimization results in severe resource underutilization and an inability to adapt to the dynamic critical path shifts inherent in multi-step reasoning.

To bridge this gap, we propose SPIMOE, the first hybrid sparse heterogeneous PIM framework tailored for long-reasoning MoE inference. By orchestrating algorithm-hardware co-design, SPIMOE

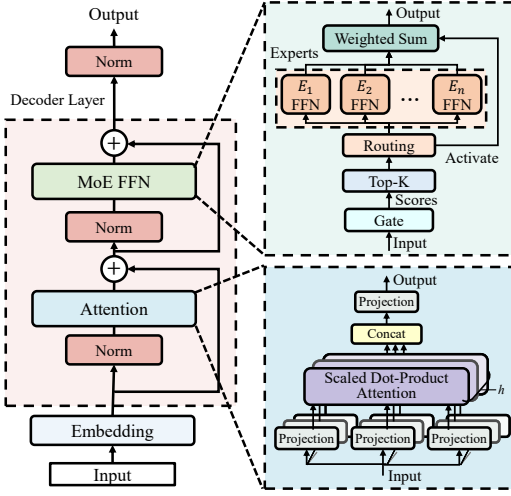


Figure 1: Model structure of MoE transformer.

improves the efficiency of both Attention and expert-FFN execution throughout the entire reasoning process. Our key contributions are as follows:

- (1) **Algorithm Level:** We propose **reasoning-aware and PIM-friendly** techniques, specifically a block-sparse attention mechanism and an adaptive expert routing strategy. These techniques integrate sparse block selection with physical KV cache eviction to prune redundant blocks at semantic CoT boundaries, and utilizes adaptive expert routing with dynamic thresholds and thinking-critical expert boosting to balance HBM-PIM channel workloads.
- (2) **Hardware Level:** We design a heterogeneous SRAM-PIM and HBM-PIM system with decoupled attention/FFN data paths. We further introduce static expert mapping and dynamic sub-batch scheduling to alleviate expert load imbalance and interconnect bottleneck.
- (3) We build an end-to-end system-level evaluation framework for SPIMOE. Compared with an NVIDIA A100 GPU baseline, SPIMOE achieves up to **8.35×** speedup at batch size 8 and **6.27×** speedup at sequence length 2K. Compared with the prior PIMoE design, SPIMOE delivers **3.33×** speedup for expert FFN execution.

2 Background and Motivations

2.1 MoE Architecture

MoE has emerged as a representative sparse scaling architecture in modern LLMs [5, 6, 22]. As illustrated in Fig. 1, each MoE layer comprises a router and multiple expert FFNs. A lightweight router selects the top-k experts for each token, and only the corresponding expert FFNs are executed. This token-wise sparse activation mechanism improves model scalability, but it also makes expert computation highly dependent on routing decisions. As LLM applications increasingly shift from short-response generation to long-chain reasoning, such routing-dependent execution exhibits substantially different runtime characteristics.

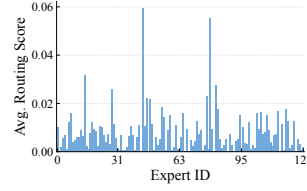


Figure 2: Average routing score distribution across experts for thinking-critical tokens.

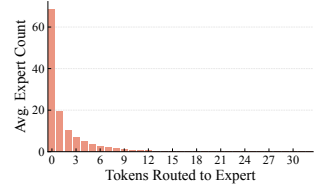


Figure 3: Distribution of tokens routed to each expert per decode step with batch size 32.

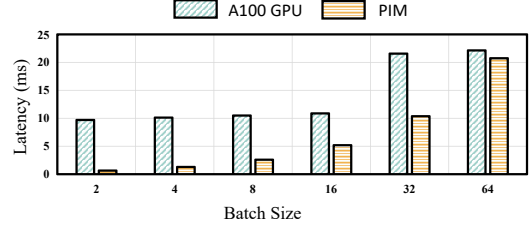


Figure 4: MoE FFN execution time comparison under enforced identical routing decisions across tokens.

During prolonged decoding, the continuous growth of the KV cache makes attention increasingly dominant in runtime, as it becomes more constrained by storage capacity and memory accesses. Meanwhile, expert activation varies across tokens, layers, and reasoning steps, leading to stronger sparsity, irregular expert-weight accesses, and dynamic load imbalance. As shown in Fig. 2, the average routing scores of reasoning-critical tokens are highly skewed across experts, indicating that only a small subset of experts consistently receives high importance. Moreover, Fig. 3 shows a clear long-tail distribution in the number of tokens routed to each expert at every decode step. Most experts receive only a few tokens or are not activated at all. These observations reveal strong temporal sparsity and severe expert load imbalance in long-reasoning MoE inference. Taken together, they suggest that long-reasoning MoE inference is increasingly constrained by KV cache-intensive attention and irregular expert execution, rather than raw compute alone. As a result, such workloads are difficult to accelerate efficiently on conventional architectures.

2.2 PIM Architecture

PIM has emerged as a promising approach to alleviating the memory wall by integrating computation into memory, making it particularly suitable for memory-bound workloads [23–25]. This property benefits both major components of long-reasoning MoE inference. For attention, as discussed in Sec. 2.1, long-chain reasoning continuously enlarges the KV cache, which naturally favors near-data execution. For expert FFNs, long-reasoning MoE exhibits stronger sparsity and a more long-tailed expert activation pattern. As a result, expert-FFN execution typically appears as a small-batch, narrow GEMM or GEMV workload. Such workloads are difficult to execute efficiently on GPUs, as GPU architectures are primarily optimized for larger and denser matrix operations, as illustrated in Fig. 4. Although existing PIM-based studies have demonstrated the potential of near-data execution for MoE [17–19] and sparse

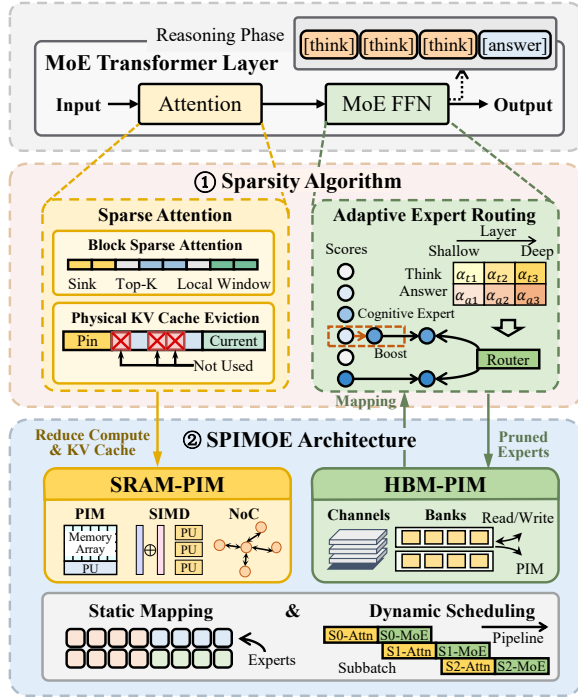


Figure 5: Overview of SPIMOE framework.

workloads [20, 21, 26], most of them do not explicitly target long-reasoning MoE inference. Moreover, the reliance on homogeneous execution substrates prevents prior designs from efficiently accommodating the divergent bottlenecks of attention and expert FFN under long-reasoning MoE inference.

2.3 Motivations of SPIMOE

The above analysis reveals three key design requirements. First, attention and expert FFNs exhibit different access patterns and bottlenecks, favoring a heterogeneous PIM architecture with decoupled data paths. Second, highly skewed and long-tailed expert activation patterns result in severe load imbalance and redundant expert execution. Such characteristics are difficult to address through hardware scheduling alone, thereby motivating joint support for architecture-level load balancing and algorithm-level adaptive expert routing. Third, KV cache growth makes attention increasingly dominant, motivating reasoning-aware sparse attention that reduces both computation and physical cache capacity without degrading reasoning quality. **These observations collectively motivate a hybrid heterogeneous PIM framework with joint algorithm-hardware optimization for long-reasoning MoE inference.**

3 SPIMOE Framework

3.1 Framework Overview

SPIMOE is an algorithm-architecture co-design framework that maps long-reasoning MoE inference onto a heterogeneous PIM, as shown in Fig. 5. It adopts Attention-FFN Disaggregation (AFD): SRAM-PIM executes Attention, while HBM-PIM handles QKV projections and expert FFNs. Adaptive expert routing dynamically



(a) First layer.

(b) Last layer.

Figure 6: Routing score distribution across experts. Each sub-plot shows important (think), unimportant (think), and post-think routing scores.

prunes low-contribution experts based on the reasoning phase and layer depth, reducing FFN computation and mitigating channel congestion in HBM-PIM. Block sparse attention combined with reasoning-aware KV cache eviction reduces per-step attention cost and reclaims capacity for longer reasoning sequences. To further improve utilization, SPIMOE adopts a dynamic sub-batch scheduling strategy that partitions the batch into smaller sub-batches, allowing attention and MoE FFN to execute in an overlapped pipeline across consecutive sub-batches.

3.2 Sparsity Algorithms

3.2.1 Adaptive Expert Routing Algorithm.

Routing Weight Analysis. Fig. 6 shows the routing score distribution across experts at different layers. The distributions are highly skewed, with a few experts dominating, indicating substantial sparsity. This skewness increases with depth, as the distributions in deeper layers become more concentrated. The distributions also vary across reasoning phases, with routing concentrating on a few dominant experts during the answer phase. Notably, important reasoning tokens tend to focus on a small subset of critical experts, a phenomenon also observed in prior work [27], where these experts are referred to as *Cognitive Experts*. These observations motivate our adaptive routing strategy.

Phase-Depth Alpha Partitioning. We define a pruning coefficient $\alpha_{p,r}$, where $p \in \{\text{think}, \text{answer}\}$ denotes the reasoning phase detected online via the $\langle \text{think} \rangle$ token, and $r \in \{\text{shallow}, \text{middle}, \text{deep}\}$ denotes the layer depth tier. At layer $l \in \{1, \dots, L\}$ where L denotes the total number of layers, only experts whose routing weight satisfies

$$w_i^{(l)} \geq \alpha_{p,r} \cdot \max_{j \in \text{Top-}K} w_j^{(l)} \quad (1)$$

are retained, with a minimum count $k_{\min}$ enforced. The answer phase permits higher α values due to its more concentrated weight distribution.

Cognitive Expert Boost. For each layer l , we identify the top- N_{cog} cognitive experts $\mathcal{E}_l^{\text{cog}}$ through offline profiling of normalized

Algorithm 1 Adaptive expert routing.

Require: Routing weights $\{w_i^{(l)}\}$, phase p , depth tier r , cognitive expert set $\mathcal{E}_l^{\text{cog}}$, expert mapping $\mathcal{P}$, boost β , thresholds $\{\alpha_{p,r}\}$, minimum $k_{\min}$

Ensure: Pruned expert set S_{prune} and renormalized weights

- 1: **for** each expert i in Top- K **do**
- 2: $\tilde{w}_i^{(l)} \leftarrow \beta \cdot w_i^{(l)}$ if $i \in \mathcal{E}_l^{\text{cog}}$ and $p = \text{think}$, else $w_i^{(l)}$
- 3: **end for**
- 4: $\tau \leftarrow \alpha_{p,r} \cdot \max_j \tilde{w}_j^{(l)}$
- 5: $S_{\text{prune}} \leftarrow \{i : \tilde{w}_i^{(l)} \geq \tau\}$; pad to $k_{\min}$ if $|S_{\text{prune}}| < k_{\min}$
- 6: Compute per-channel load η_c from $\mathcal{P}$
- 7: **for** each $i \in S_{\text{prune}}$ on congested channel ($\eta_c > \mu_\eta + \sigma_\eta$) **do**
- 8: **if** $i \notin \mathcal{E}_l^{\text{cog}}$ **and** $|S_{\text{prune}}| > k_{\min}$ **then**
- 9: $S_{\text{prune}} \leftarrow S_{\text{prune}} - \{i\}$
- 10: **end if**
- 11: **end for**
- 12: Renormalize weights of remaining experts in S_{prune}
- 13: **return** S_{prune}

activation frequency on reasoning-critical tokens [27] from a calibration set disjoint from evaluation data. At inference time, their routing weights are amplified by a boost factor $\beta > 1$ prior to the threshold test:

$$\tilde{w}_i^{(l)} = \begin{cases} \beta \cdot w_i^{(l)}, & i \in \mathcal{E}_l^{\text{cog}}, \\ w_i^{(l)}, & \text{otherwise.} \end{cases} \quad (2)$$

The boosted weights $\tilde{w}_i^{(l)}$ then undergo Eq. (1), ensuring cognitive experts are preferentially retained under aggressive pruning.

Parameter Calibration. The phase-depth thresholds $\{\alpha_{p,r}\}$, boost factor β , and $k_{\min}$ are calibrated offline on a small calibration set disjoint from the evaluation data. A lightweight grid search selects configurations on the accuracy-performance Pareto frontier. The selected parameters are reused across datasets for the same model, while new model families require one-time recalibration due to different routing distributions.

PIM-Aware Expert Pruning. On HBM-PIM, experts on the same channel create serialized DRAM computation and NoC communication bottlenecks. Let $a_{l,e} \in \{0,1\}$ denote the activation indicator for expert (l,e) . The instantaneous load per channel is $\eta_c = \sum_{(l,e) \in \mathcal{P}(c)} a_{l,e}$, counting the number of currently active experts mapped to channel c . We prioritize pruning experts $i \notin \mathcal{E}_l^{\text{cog}}$ on congested channels ($\eta_c > \mu_\eta + \sigma_\eta$), as shown in Algorithm 1.

3.2.2 Sparse Attention Algorithm.

Attention Pattern Analysis. Fig. 7 reveals a three-stage evolution of attention patterns across network depth. In middle layers, the model selectively attends to a small subset of reasoning blocks visible as vertical stripes, while aggregation behavior emerges as summary blocks consolidate information from earlier segments. In deep layers, attention further collapses to a more local pattern with only a narrow local window and a few globally important anchor tokens. Once the thinking phase concludes, most reasoning tokens quickly become less important. This progressive sparsification indicates that the reasoning chain serves as an intermediate result rather than persistent memory, motivating both block-granularity sparse computation and physical eviction of obsolete KV entries.

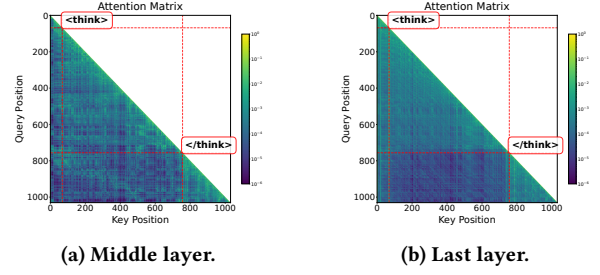


Figure 7: Attention heatmaps during reasoning at different layers.

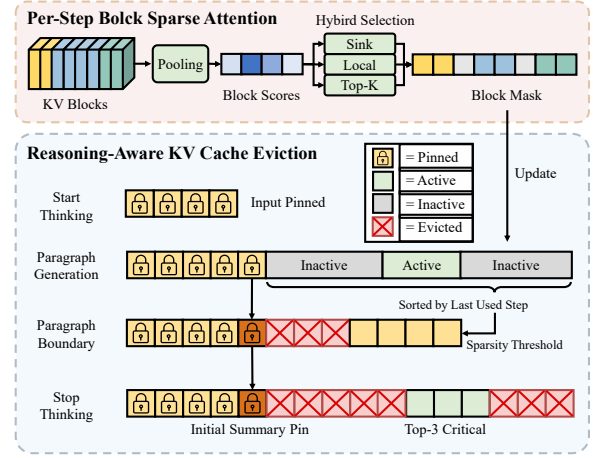


Figure 8: Design of sparse attention algorithm.

Existing methods, such as Quest [28] and MInference [29] reduce computation via token skipping or mask-based sparsity, but the skipped KV entries remain physically resident. As illustrated in Fig. 8, we propose a two-level scheme: block sparse attention for per-step computation reduction, and reasoning-aware KV cache eviction for storage savings.

Block Sparse Attention. The KV sequence of length s is partitioned into chunks of size B_{chunk} aligned with the SRAM macro row width. A representative key per chunk is obtained via mean pooling: $\bar{k}_j = B_{\text{chunk}}^{-1} \sum_{t \in \text{chunk}_j} k_t$. At each decode step, the query q computes coarse scores $s_j = q^\top \bar{k}_j$ against all $N = \lceil s/B_{\text{chunk}} \rceil$ chunk representatives, and the visible KV cache for attention is formed as:

$$\mathcal{V} = \underbrace{\mathcal{V}_{\text{sink}}}_{n_s \text{ tokens}} \cup \underbrace{\mathcal{V}_{\text{local}}}_{n_l \text{ tokens}} \cup \underbrace{\mathcal{V}_{\text{Top-K}}}_{K_b \text{ chunks by } s_j}, \quad (3)$$

where $\mathcal{V}_{\text{sink}}$ preserves prompt context as persistent global anchors, $\mathcal{V}_{\text{local}}$ maintains generation coherence, and $\mathcal{V}_{\text{Top-K}}$ captures the salient reasoning blocks. The attention output is computed exclusively over $\mathcal{V}$:

$$o = \text{Softmax}\left(qK_{\mathcal{V}}^\top / \sqrt{d_h}\right) V_{\mathcal{V}}, \quad (4)$$

reducing per-step complexity from $O(s)$ to $O(\rho s)$ under sparsity ratio ρ .

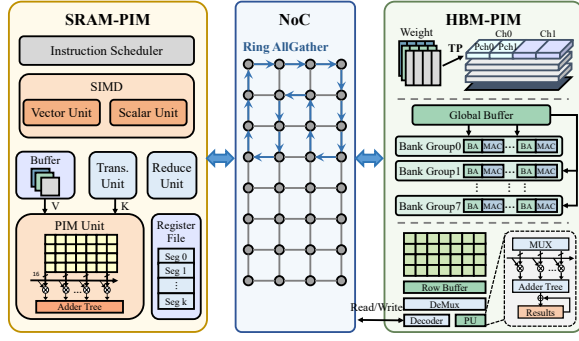


Figure 9: Overall architecture of SPIMOE.

Physical KV Cache Eviction. Block sparse attention reduces computation but not physical cache size. To achieve genuine storage savings, we introduce physical eviction triggered at semantic boundaries. A designated anchor layer records per-chunk last-used timestamps as a cross-layer importance proxy. Eviction is triggered at structural boundaries such as paragraph separators `\n\n` and the end-of-thinking marker `</think>`. If no boundary is detected for 2,048 consecutive tokens and KV cache occupancy exceeds 80%, a conservative fallback trigger is used.

Upon triggering, the cache is partitioned into three regions. **Pinned** chunks include prompt tokens and important chunks of the initial paragraph, which are permanently retained. **Current** chunks belong to the ongoing segment and are excluded from eviction. **Previous** chunks are historical eviction candidates sorted by last-used timestamps; only the top p_{evict} fraction of most recently attended chunks is retained. Evicted entries are physically removed across all layers. Evicted entries are reclaimed at block granularity and returned to a global free list, avoiding sub-page fragmentation; incremental chunk updates with reduction every B_{chunk} tokens incur $O(1/B_{chunk})$ amortized overhead. Unlike mask-based approaches that merely reduce dynamic energy, physical eviction directly reclaims scarce capacity, enabling longer reasoning within the same hardware budget.

3.3 SPIMOE Architecture

3.3.1 Overall Architecture.

The fundamental design principle is Attention-FFN Disaggregation, as shown in Fig. 9. The two dominant operator classes in MoE transformer inference are mapped onto physically distinct PIM subsystems whose characteristics match the respective computational profiles.

SRAM-PIM. The SRAM-PIM subsystem contains 32 independent cores optimized for decode-phase attention. Each core integrates a PIM unit for matrix-vector multiplication, a vector compute unit for nonlinear functions such as softmax, a scalar compute unit for scalar operations, a transpose unit for K cache transposition, and a reduce unit for Top-K computation supporting expert routing and block-sparse attention selection. The low-latency, configurable SRAM substrate is well-suited for the irregular, access-intensive attention computation pattern.

HBM-PIM. The HBM-PIM subsystem adopts the memory architecture design from Newton [30], while the in-situ MAC units integrated in each bank perform GEMV operations on resident

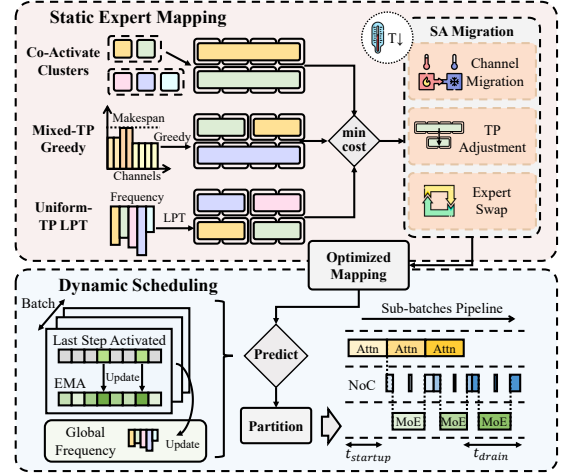


Figure 10: Design of mapping and scheduling strategy.

weight matrices without external data movement. In compute mode, the input activation vector is broadcast via a global buffer to all banks in parallel, exploiting the massive internal bandwidth of the TSV-interconnected HBM3 stack. This approach follows the characteristics of MoE inference, where each activated expert processes a few tokens on average, resulting in an operation that maps to the bank-parallel PIM execution model.

System Topology. HBM channels and SRAM cores are connected through a 4×8 NoC mesh running at 256 bits/flit and 800 MHz, with a channel-to-core mapping. Each NoC link supports single-directional communication. The per-layer dataflow alternates between HBM-PIM and SRAM-PIM, enabling pipelined overlap across consecutive sub-batches as detailed in Section 3.4.2.

3.3.2 Dataflow and Communication Design.

Per-Layer Operator Pipeline. Each transformer layer is decomposed into 17 operator stages executed alternately on SRAM-PIM and HBM-PIM, with five NoC communication phases interleaved: KV all-gather before attention, activation all-gather before projection and before FFN1, TP communication around FFN2, and MoE output gather after FFN2.

Two modes are supported depending on how the FFN2 weight matrix is partitioned. In *row mode*, each TP shard produces a partial output that is summed via all-reduce after FFN2 ($2(t-1)$ ring rounds, data size d_{size}). In *col mode*, each shard requires the full intermediate activation, necessitating an all-gather before FFN2 ($t-1$ rounds, data size d_{inter}/n_{act}). The mode is selected based on the model's intermediate-to-model dimension ratio.

After FFN2, expert results are aggregated in three sub-phases: (1) TP all-reduce within each expert's shard group (row mode only); (2) inter-expert all-gather across the first cores of all activated experts; (3) result scatter to distribute the final output.

Ring Topology Optimization. Ring all-gather and all-reduce latency is dominated by the maximum bidirectional hop count in the ring. Given n participating cores on the 4×8 mesh, we find a Hamiltonian cycle with ring order $\pi = (\pi_1, \dots, \pi_n)$ that minimizes $\max_i [\text{hop}(\pi_i, \pi_{i+1}) + \text{hop}(\pi_{i+1}, \pi_i)]$, where $\pi_{n+1} = \pi_1$. For rectangular sub-meshes, a snake-order traversal yields optimal rings in $O(n)$ time; for general core sets, a greedy nearest-neighbor construction

followed by 2-opt local search is applied. Optimized ring orders are cached per core set to avoid recomputation.

3.4 Mapping and Scheduling Mechanism

3.4.1 Static Expert Mapping.

The static assignment of experts to HBM channels determines load balance and communication cost. An Integer Linear Programming (ILP) formulation over all layers, experts, channels, and TP factors yields an intractable search space; we instead adopt a multi-candidate heuristic strategy with simulated annealing refinement, as shown in Fig. 10.

Co-activation Clustering. From profiled activation data, we compute the co-activation ratio $\gamma_{ij} = \text{co_act}(i, j) / \min(\text{cnt}_i, \text{cnt}_j)$ for each expert pair, where $\text{co_act}(i, j)$ is the co-activation count of experts i and j , and cnt_i is the activation count of expert i . Experts frequently activated by the same tokens are grouped into cliques via greedy maximal-clique extraction. Placing clique members on non-overlapping channel groups improves compute load balancing and enables parallel execution. The clique size determines the TP factor and channel group assignment for each member.

Multi-Candidate Competition. For each layer, three candidate mapping strategies are evaluated: (1) co-activation clustering with topology-aware channel assignment; (2) mixed-TP greedy, where each expert independently selects the (TP, channel group) pair minimizing its cost; and (3) uniform-TP with Longest Processing Time First (LPT) load balancing across channel groups. The candidate with the lowest per-step cost $\mathcal{J} = \mathcal{J}_{\text{DRAM}} + \mathcal{J}_{\text{NoC}}$ is selected, where $\mathcal{J}_{\text{DRAM}}$ is the channel computation makespan and $\mathcal{J}_{\text{NoC}}$ is the total NoC communication cost.

SA Migration. The selected mapping is further refined via Simulated Annealing (SA) with three neighborhood operators: expert migration to another channel group, TP factor adjustment, and expert swap. The cost function is evaluated per decode step using profiled activation patterns, with incremental NoC updates for affected tokens only.

3.4.2 Dynamic Scheduling.

Static expert mapping optimizes the average case but cannot adapt to runtime token-to-expert assignments. Dynamic sub-batch scheduling partitions the batch so that each sub-batch's MoE execution time approximates its attention time, improving pipeline efficiency, as shown in Fig. 10.

Cost Prediction. Since gate routing results are unavailable until after QKV projection, MoE cost must be predicted before attention begins. At step 0, a lookup table provides cost estimates from offline profiling. At subsequent steps, the predictor constructs a gating score matrix $\mathbf{G} \in \mathbb{R}^{L \times N_E}$:

$$\mathbf{G} = \lambda_{\text{prev}} \odot \mathbf{M}_{\text{prev}} + \lambda_{\text{ema}} \odot \hat{\mathbf{A}} + \lambda_{\text{freq}} \odot \mathbf{F}, \quad (5)$$

where $\mathbf{M}_{\text{prev}}$ is the previous step's activation mask, $\hat{\mathbf{A}}$ is an Exponential Moving Average (EMA) of per-token expert affinity, and $\mathbf{F}$ is a global frequency prior. The blending coefficients λ_{prev} , λ_{ema} , and λ_{freq} weight $\mathbf{M}_{\text{prev}}$, $\hat{\mathbf{A}}$, and $\mathbf{F}$, respectively, and are calibrated per layer from measured step-to-step autocorrelation. The top- K experts are selected from $\mathbf{G}$, and the predicted MoE cost is obtained via the expert-to-core mapping table.

Table 2: Model configurations.

Model	Params	L	d	d_{inter}	N_E	K	H_Q	H_{KV}	d_h
Qwen3-30B-A3B	30B	48	2048	6144	128	8	32	4	128
Phi-mini-MoE	15B	32	4096	960	16	2	32	8	128
Switch-Large-128	24B	12	1024	4096	128	1	16	16	64
Switch-Base-16	0.9B	6	768	3072	16	1	12	12	64

Sub-batch Partition. Sub-batches are formed via greedy token packing and closed when the accumulated predicted MoE cost of tokens in the sub-batch exceeds the attention time threshold determined by the sub-batch size.

Pipeline Execution. An overlap execution mode interleaves attention and MoE across sub-batches: SRAM-PIM computes attention for sub-batch $bi + 1$ while HBM-PIM executes MoE for sub-batch bi . The overall step latency is:

$$T_{\text{step}} = T_{\text{startup}} + \sum_{bi} \max(T_{\text{attn}}^{(bi)}, T_{\text{moe}}^{(bi)}) + T_{\text{drain}}, \quad (6)$$

where T_{startup} and T_{drain} account for the first and last sub-batch that cannot be overlapped.

4 Experiment

4.1 Experimental Setup

Models. Qwen3-30B-A3B [7] and Phi-mini-MoE-instruct [8] are used for end-to-end evaluation. For component-level comparison with PIMoE, we use Switch-Large-128 [5] and Switch-Base-16. We additionally use the dense Qwen3-1.7B model for sparse-attention evaluation. In Table 2, L denotes the number of layers, d denotes the model hidden size, and d_{inter} denotes the total intermediate size of the FFN module. H_Q and H_{KV} denote the number of query heads and key-value heads, respectively, and d_h is the hidden dimension per head. N_E denotes the total number of experts, and K denotes the number of activated experts per token.

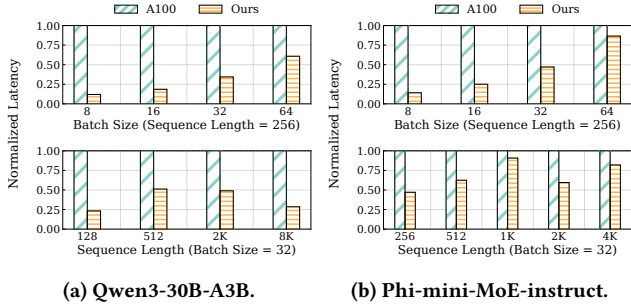
Benchmarks. We evaluate on five reasoning benchmarks with varying difficulty levels. For mathematical reasoning, GSM8K [31] covers grade-school arithmetic, MATH-500 [32] covers high-school competition problems, and AIME 2024 [33] covers competition-level challenges. For science reasoning, GPQA [34] covers graduate-level questions, while ARC-Challenge [35] covers grade-school science logic. We enable reasoning for all accuracy evaluations.

Hardware Specification. The hardware configuration of the proposed SPIMOE architecture is detailed in Tab. 3. The system integrates 32 SRAM-PIM cores with 4 HBM-PIM modules, interconnected via a 2.5D silicon interposer. Each SRAM-PIM core operates at 800 MHz, while each HBM-PIM bank integrates a 400 MHz PU delivering 6.4 GFLOPS for FP16 GEMV. The four HBM3 modules provide a total capacity of 96 GB, sufficient to accommodate MoE models such as Qwen3-30B-A3B.

Baselines. We use an NVIDIA A100-80GB GPU running standard inference as the baseline. For sparse attention, we compare against two representative methods, MInference [29] and Quest [28]. For MoE acceleration on PIM architectures, we compare with PIMoE [19] at the MoE FFN component level, since PIMoE does not support attention or KV cache management.

Table 3: Hardware configuration.

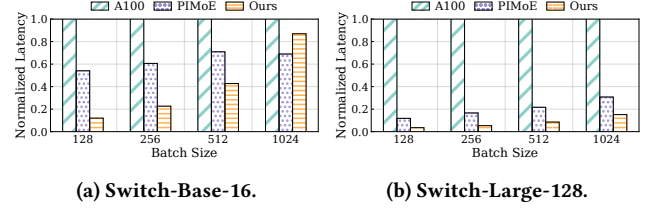
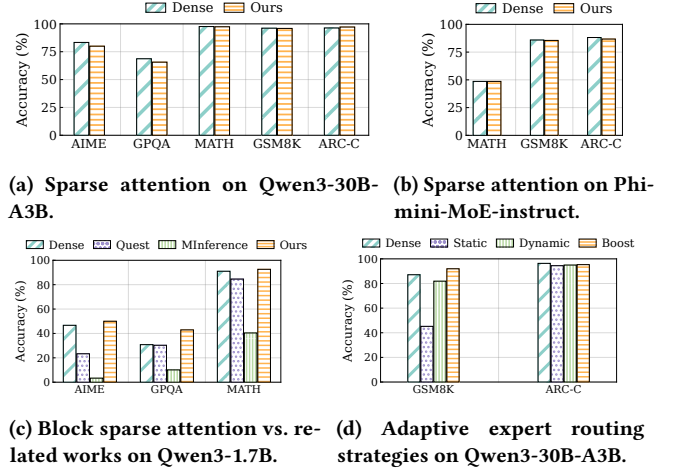
SPIMOE	Composition	32 SRAM-PIM core, 4 HBM-PIM
SRAM-PIM	Memory Configuration	384KB Activation Memory, 32KB Temp Memory
	PIM Unit	800MHz, 16 Macro Groups, 16 PUs per MG, 12.8 GFLOPS per PU
HBM-PIM	Memory Configuration	HBM3, 24GB/HBM, 8 dies, 8 DRAM per die, 2 Channel per DRAM, 2 Pseudo Channels per Channel, 4 Bank Groups per pCH, 8 Banks per BG
	Processing Unit (PU)	400MHz, 1 PU per Bank, 6.4 GFLOPS per PU

**Figure 11: Normalized latency vs. A100 GPU baseline.**

Simulation Infrastructure. Our simulation infrastructure builds upon the framework developed for our prior heterogeneous PIM work, HPIM [36], and is extended to model SPIMOE-specific mechanisms. The SRAM-PIM subsystem is modeled using our in-house CIMFlow framework [37], with digital modules implemented in Verilog HDL and synthesized using Synopsys Design Compiler at 12 nm for area and power estimation. The HBM-PIM subsystem follows the bank-level PIM organization of Newton [30] and is modeled using an extended DRAMsim3 [38] under the HBM3 specification [39]; its PUs and global buffers are also synthesized at 12 nm. The 4×8 mesh NoC is modeled using an extended Noxim [40].

4.2 Performance Evaluation

End-to-End Speedup. Fig. 11 presents the end-to-end speedup under two sweep dimensions. At small batch sizes where MoE FFN dominates the execution time, the heterogeneous PIM architecture achieves substantial acceleration: at batch size 8 with sequence length 256, Qwen3-30B-A3B achieves $8.35\times$ speedup, and Phi-mini-MoE-instruct achieves $7.08\times$ speedup over the A100 baseline. As batch size increases, the per-expert token count grows, and GPU GEMM utilization improves, reducing the performance gap with the GPU baseline; at batch size 64, the speedup diminishes as the workload transitions from bandwidth-bound to compute-bound. Meanwhile, the sequence-length sweeps in Fig. 11 show that, at a fixed batch size of 32, speedup initially decreases as the sequence length increases due to the growing attention cost. It recovers at longer sequences as block-sparse attention reduces the SRAM-PIM workload. For Qwen3-30B-A3B at 8K tokens, SPIMOE achieves a $3.51\times$ speedup, demonstrating the effectiveness of the proposed co-design for long-reasoning workloads.

**Figure 12: Comparison with PIMoE [19] (normalized to A100 GPU baseline).****Figure 13: Accuracy evaluation results.**

MoE FFN Component-level Speedup. Since PIMoE [19] targets MoE FFN acceleration, Fig. 12 compares the MoE FFN execution of SPIMOE against both PIMoE and A100 under identical routing decisions. FFN latency includes both computation and communication, excluding latency hidden by pipeline overlap, and is normalized to A100. SPIMOE consistently outperforms both baselines; on Switch-Large-128, it achieves from $3.33\times$ to $2.03\times$ speedup over PIMoE as the batch size increases from 128 to 1024.

Hardware Overhead. The synthesized SRAM-PIM subsystem occupies 183.2 mm^2 and consumes 0.787 W , while the added compute and buffer logic in HBM-PIM occupies 0.268 mm^2 area and consumes 9.599 W . Power is evaluated on Qwen3-30B-A3B with batch size 8 and context length 256. Thermal analysis shows a steady-state HBM-PIM temperature of approximately 46.7°C during GEMV execution.

4.3 Accuracy Evaluation

Block-sparse Attention. Fig. 13 evaluates the accuracy of block-sparse attention. Figs. 13a and 13b show that block-sparse attention preserves reasoning accuracy across two MoE models and five datasets while reducing the KV cache by approximately 50% through physical eviction. Since Phi-mini-MoE-instruct supports a maximum output length of 4K tokens, AIME and GPQA are omitted for this model. On Qwen3-1.7B (Fig. 13c), our method maintains accuracy close to full attention and outperforms Quest [28] and MInference [29]. We use a sink size of 128 tokens, a local window of 128 tokens, and an 80% sparsity ratio, and enable block-sparse

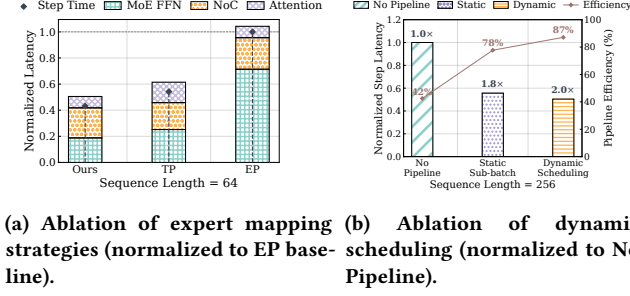


Figure 14: Ablation studies on expert mapping and dynamic scheduling.

attention only for sequences longer than 1K tokens to ensure that at least one block is retained.

Adaptive Expert Routing. Fig. 13d evaluates three expert routing strategies on Qwen3-30B-A3B with a 2K output budget. *Static* applies a global uniform pruning coefficient α across all layers and phases. *Dynamic* employs depth-stratified coefficients with higher minimum expert counts in shallow layers and reduced thresholds during the thinking phase. *Boost* extends the dynamic strategy by additionally boosting the cognitive experts during the thinking phase. The static strategy leads to severe accuracy degradation as it removes experts critical for reasoning without distinction. The cognitive-enhanced strategy preserves reasoning quality and achieves accuracy comparable to the dense baseline, confirming that reinforcing reasoning-critical experts is important under aggressive expert routing.

4.4 Ablation Studies

Expert Mapping. Fig. 14a compares expert mapping strategies, with all latencies normalized to the EP baseline. The optimized two-phase TP+EP mapping achieves 1.89x speedup.

Dynamic Scheduling. Fig. 14b evaluates the impact of sub-batch pipelining and dynamic scheduling. Compared to a non-pipelined baseline, static sub-batch partitioning achieves 1.80x speedup with 77.7% pipeline efficiency, while the dynamic scheduling further improves to 1.99x speedup and 87.3% pipeline efficiency.

NoC Topology Optimization. Fig. 15c shows the per-operator latency reduction from NoC topology optimization. All-gather and all-reduce operations each achieve 3.12x speedup through conflict-free scheduling on the NoC mesh. The NoC optimization yields a 1.16x overall speedup in the end-to-end test. Fig. 15 visualizes the NoC ring topologies before and after optimization, showing how the NoC topology optimization algorithm redistributes communication paths to eliminate link conflicts and reduce the maximum number of serialized gather rounds.

Sparse Attention. Fig. 16a evaluates the impact of block sparse attention on end-to-end latency under fixed batch size 8 with varying sequence lengths. As sequence length increases beyond 1K, the sparse attention mechanism progressively reduces the SRAM-PIM workload. At 8K tokens, the system with sparse attention achieves 2.36x speedup, while the system without sparse attention becomes 1.47x slower than the A100 baseline. This demonstrates that block sparse attention is essential for maintaining PIM acceleration at long sequences.

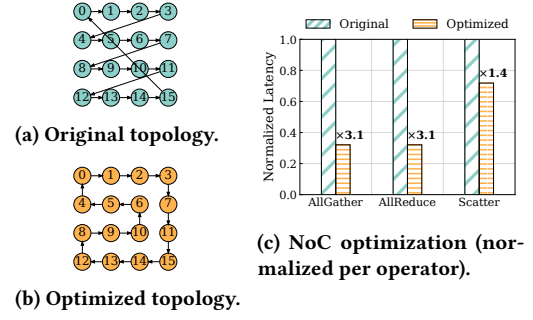


Figure 15: NoC ring topology comparison and optimization ablation.

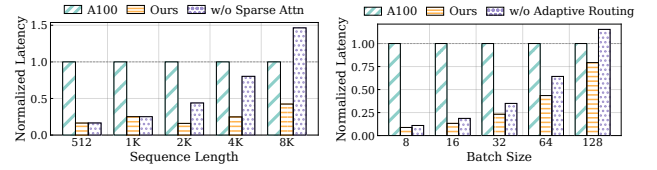


Figure 16: Ablation studies on sparse attention and adaptive expert routing.

Adaptive Expert Routing. Fig. 16b compares the system with and without adaptive expert routing at short sequence lengths where MoE FFN dominates execution time. Without adaptive routing, all top-K experts are activated, increasing the FFN computation. With adaptive routing, the system selectively prunes redundant experts, achieving a 1.53x speedup with batch size 128. The performance improvement is observed across different batch sizes, confirming that adaptive expert routing complements the architectural advantage of PIM by reducing the effective computation per token.

5 Conclusion

In this paper, we present SPIMOE, the first hybrid-sparse heterogeneous PIM framework for efficient long-chain reasoning inference in MoE models. SPIMOE jointly addresses KV cache-intensive attention and highly sparse, irregular expert-FFN execution through reasoning-aware algorithm-hardware co-design. At the algorithm level, it integrates adaptive expert routing and block-sparse attention with KV cache eviction to eliminate redundant expert computation, attention computation, and KV-cache storage. Architecturally, SPIMOE realizes Attention-FFN Disaggregation on a heterogeneous PIM architecture integrating SRAM-PIM and HBM-PIM, effectively minimizing both data movement and storage overhead. Experimental results show that SPIMOE achieves up to 8.35x end-to-end speedup over an NVIDIA A100 GPU and 3.33x speedup in MoE FFN execution over PIMoE, while maintaining reasoning accuracy comparable to full-attention baselines. These results demonstrate that the synergy between reasoning-aware sparsity and heterogeneous PIM design is a promising direction for scaling future sparse LLM inference systems.

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* 30 (2017).
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [3] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [4] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [5] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [6] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668* (2020).
- [7] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [8] Zichong Li, Chen Liang, Zixuan Zhang, Ilgee Hong, Young Jin Kim, Weizhu Chen, and Tuo Zhao. 2025. Slimmoe: Structured compression of large moe models via expert slimming and distillation. *arXiv preprint arXiv:2506.18349* (2025).
- [9] Damai Dai, Chengqi Deng, Chenggang Zhao, R X Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y Wu, et al. 2024. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066* (2024).
- [10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shitong Ma, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [11] Qiguang Chen, Libo Qin, Jinhao Liu, Dengyun Peng, Jiannan Guan, Peng Wang, Mengkang Hu, Yuhang Zhou, Te Gao, and Wanxiang Che. 2025. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models. *arXiv preprint arXiv:2503.09567* (2025).
- [12] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [13] [n.d.]. Learning to Reason with LLMs. <https://openai.com/index/learning-to-reason-with-llms/>
- [14] Cenlin Duan, Jianlei Yang, Yiou Wang, Yikun Wang, Yingjie Qi, Xiaolin He, Bonan Yan, Xueyan Wang, Xiaotao Jia, and Weisheng Zhao. 2024. Towards efficient sram-pim architecture design by exploiting unstructured bit-level sparsity. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [15] Cenlin Duan, Jianlei Yang, Yikun Wang, Yiou Wang, Yingjie Qi, Xiaolin He, Bonan Yan, Xueyan Wang, Xiaotao Jia, and Weisheng Zhao. 2025. Efficient SRAM-PIM co-design by joint exploration of value-level and bit-level sparsity. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 45, 1 (2025), 246–259.
- [16] Yingjie Qi, Jianlei Yang, Rubing Yang, Cenlin Duan, Xiaolin He, Ziyang He, Weitao Pan, and Weisheng Zhao. 2025. CIMinus: Empowering Sparse DNN Workloads Modeling and Exploration on SRAM-Based CIM Architectures. *IEEE Trans. Comput.* 75, 1 (2025), 380–394.
- [17] Yue Pan, Zihan Xia, Po-Kai Hsu, Lanxiang Hu, Hyungyo Kim, Janak Sharda, Minxuan Zhou, Nam Sung Kim, Shimeng Yu, Tajana Rosing, et al. 2025. Stratum: System-Hardware Co-Design with Tiered Monolithic 3D-Stackable DRAM for Efficient MoE Serving. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 1–17.
- [18] Sungmin Yun, Kwanhee Kyung, Juhwan Cho, Jaewan Choi, Jongmin Kim, Byeongho Kim, Sukhan Lee, Kyomin Sohn, and Jung Ho Ahn. 2024. Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1429–1443.
- [19] Lizhou Wu, Haozhe Zhu, Siqi He, Xuanda Lin, Xiaoyang Zeng, and Chixiao Chen. 2025. PIMoE: Towards efficient MoE transformer deployment on NPU-PIM system through throttle-aware task offloading. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7.
- [20] Fengbin Tu, Zihan Wu, Yiqi Wang, Ling Liang, Liu Liu, Yufei Ding, Leibo Liu, Shaojun Wei, Yuan Xie, and Shouyi Yin. 2022. TranCIM: Full-digital bitline-transpose CIM-based sparse transformer accelerator with pipeline/parallel reconfigurable modes. *IEEE Journal of Solid-State Circuits* 58, 6 (2022), 1798–1809.
- [21] Zizhuo Fu, Xiaotian Guo, Wenxuan Zeng, Shuzhang Zhong, Yadong Zhang, Peiyu Chen, Runsheng Wang, Le Ye, and Meng Li. 2025. H2EAL: Hybrid-Bonding Architecture with Hybrid Sparse Attention for Efficient Long-Context LLM Inference. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [22] Weilin Cai, Juyong Jiang, Fan Wang, Jing Tang, Sunghun Kim, and Jiayi Huang. 2025. A survey on mixture of experts in large language models. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [23] Jaehyun Park, Jaewan Choi, Kwanhee Kyung, Michael Jaemin Kim, Yongsuk Kwon, Nam Sung Kim, and Jung Ho Ahn. 2024. Attacc! unleashing the power of pim for batched transformer-based generative model inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 103–119.
- [24] Minseok Seo, Xuan Truong Nguyen, Seok Joong Hwang, Yongkee Kwon, Guhyun Kim, Chanwook Park, Ilkon Kim, Jaehun Park, Jeongbin Kim, Woojae Shin, et al. 2024. Ianus: Integrated accelerator based on npu-pim unified memory system. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 545–560.
- [25] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 722–737.
- [26] Zehao Fan, Garrett Gagnon, Zhenyu Liu, and Liu Liu. 2025. Sparse Attention Remapping with Clustering for Efficient LLM Decoding on PIM. *arXiv preprint arXiv:2505.05772* (2025).
- [27] Mengru Wang, Xingyu Chen, Yue Wang, Zhiwei He, Jiahao Xu, Tian Liang, Qizhi Liu, Yunzhi Yao, Wenxuan Wang, Ruotian Ma, et al. 2025. Two experts are all you need for steering thinking: Reinforcing cognitive effort in moe reasoning models without additional training. *arXiv preprint arXiv:2505.14681* (2025).
- [28] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. Quest: Query-aware sparsity for efficient long-context LLM inference. *arXiv preprint arXiv:2406.10774* (2024).
- [29] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H Abdi, Dongsheng Li, Chin-Yew Lin, Yuyang Yang, and Lili Qiu. 2024. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. *arXiv preprint arXiv:2407.02490* (2024).
- [30] Mingxuan He, Choungki Song, Ilkon Kim, Chunseok Jeong, Seho Kim, Il Park, Mithuna Thottethodi, and TN Vijaykumar. 2020. Newton: A DRAM-maker’s accelerator-in-memory (AiM) architecture for machine learning. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 372–385.
- [31] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).
- [32] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874* (2021).
- [33] MAA Committees. [n.d.]. AIME Problems and Solutions. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions.
- [34] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. 2024. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*.
- [35] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457* (2018).
- [36] Cenlin Duan, Jianlei Yang, Rubing Yang, Yikun Wang, Yiou Wang, Lingkun Long, Yingjie Qi, Xiaolin He, Ao Zhou, Xueyan Wang, et al. 2026. Hpim: Heterogeneous processing-in-memory-based accelerator for large language models inference. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2026).
- [37] Yingjie Qi, Jianlei Yang, Yiou Wang, Yikun Wang, Dayu Wang, Ling Tang, Cenlin Duan, Xiaolin He, and Weisheng Zhao. 2025. Cimflow: An integrated framework for systematic design and evaluation of digital cim architectures. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7.
- [38] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. 2020. DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator. *Computer Architecture Letters (CAL)* 19, 2 (2020), 106–109.
- [39] 2022. High Bandwidth Memory DRAM (HBM3). *JEDDEC* (2022).
- [40] Vincenzo Catania, Andrea Mineo, Salvatore Monteleone, Maurizio Palesi, and Davide Patti. 2015. Noxim: An open, extensible and cycle-accurate network on chip simulator. In *2015 IEEE 26th international conference on application-specific systems, architectures and processors (ASAP)*. IEEE, 162–163.